

Project (Part 1)

The part 1 of the project will focus on converting raw C code to an optimized call graph. You need to perform these tasks on the C codebase in two datasets.

[Dataset 1 \(SV COMP\)](#)

[Dataset 2 \(Codenet\)](#)

Phase 1: The Foundation (C to CFG)

Don't write a C parser from scratch. Instead, use a frontend tool.

- **Option A:** Use the **LLVM/Clang** frontend. You can write a Clang RecursiveASTVisitor to traverse the AST and build your graph.
- **Option B:** Use **Python with the pycparser library**. This is much faster to prototype.
- **The Goal:** Convert the code into **Basic Blocks**. A Basic Block is a sequence of instructions with one entry point (no jumps in) and one exit point (no jumps out except at the end).

Phase 2: Static Analysis

Once you have your graph (nodes = basic blocks, edges = jumps/branches), implement one of the following analyses:

1. Reaching Definitions Analysis

- **The Goal:** For every variable at every point in the program, determine which "definitions" (assignments) of that variable could reach that point.
- **Use Case:** Identifying **uninitialized variables**. If a point in the CFG uses x, but no definition of x reaches it, you've found a potential bug!

2. Live Variable Analysis

- **The Goal:** Determine if the value currently held in a variable will be used in the future.
- **Use Case:** This is essential for **Dead Code Elimination**. If a variable is assigned a value but is never "live" again before being reassigned or the program ends, that assignment is useless.

Phase 3: Optimizations

Now that your analysis tells you what the code is doing, you can change it to be better.

Optimization	Description	Requirement
Constant Folding	Replacing $x = 3 + 5$ with $x = 8$.	AST/CFG traversal.
Constant Propagation	If $x = 10$, replace all future uses of x with 10 (until x is changed).	Reaching Definitions Analysis.
Dead Code Elimination	Removing nodes or instructions that have no effect on the program output.	Live Variable Analysis.
Unreachable Code Removal	Removing CFG nodes that have no incoming path from the "Start" node.	Simple Graph Traversal (DFS/BFS).

Bonus Optimizations

1. **Loop Detection:** Use **Dominator Trees** to find loops in the CFG and suggest "Loop-Invariant Code Motion" (moving calculations that don't change inside a loop to just before the loop starts).
2. **Web Dashboard:** Create a simple Streamlit or Flask app where a user pastes C code on the left, and the optimized CFG is rendered on the right.
3. **Security Scan:** Perform a "Taint Analysis." Mark user input (like scanf) as "tainted" and see if that tainted data can reach a "sink" (like system() or printf()) without being sanitized.

Suggested Tech Stack

- **Language:** Python (easiest for graph manipulation) or C++ (if you want to use LLVM).
- **Graph Library:** **NetworkX** (Python). It handles the nodes and edges and can even run graph algorithms for you.
- **Visualization:** **Graphviz (DOT format)**. You can export your CFG as a .dot file and render it as a PDF or PNG to see your results visually.